

actually supports it. One way to determine whether your programming language's compiler or interpreter implements this feature is to write a tail recursive factorial function and try to calculate the factorial of 100,000. If the program crashes, tail call optimization is not implemented.

Personally, I take the stance that the tail recursion technique should never be used. As stated in Chapter 2, any recursive algorithm can be implemented with a loop and a stack. Tail call optimization prevents stack overflows by effectively removing the use of the call stack. Therefore, all tail recursive algorithms can be implemented with a loop alone. Since the code for loops is much simpler than a recursive function, a loop should be used wherever tail call optimization could be employed.

Additionally, potential problems exist even if tail call optimization is implemented. Since tail recursion is possible only when the last action of a function is returning the recursive call's return value, it's impossible to do tail recursion for algorithms that require two or more recursive calls. For example, take the Fibonacci sequence algorithm calls `fibonacci(n - 1)` and `fibonacci(n - 2)`. While tail call optimization can be performed for the latter recursive call, the first recursive call will cause a stack overflow for large-enough arguments.